

```

# class wurde erst am ende programmiert, deswegen nicht oft verwendet
# init wird aufgerufen wenn neue Instanz erstellt wird
# self steht für die aktuelle Instanz
class Spieler:
    def __init__(self, name):
        self.name = name
        self.lebenspunkte = 100
        self.gold = 0
        self.istLebendig = True

spieler1 = Spieler("julia")
spieler2 = Spieler("leo")

spielerName = "julia"
lebenspunkte = 100
gold = 25
istLebendig = True

# wenn man wissen will welcher Datentyp
print(type(lebenspunkte))

lebenspunkte = 30

# in C++ wäre es else if, in python elif
if lebenspunkte <= 0:
    pass
elif lebenspunkte < 50:
    print("verletzt")
else:
    print("gut dabei")

lebenspunkte = 5
for i in range(lebenspunkte):
    print(i)

while lebenspunkte > 0:
    print("du lebst noch")
    lebenspunkte -= 1

#da true immer wahr ist läuft schleife bis break
while True:
    eingabe = input("Befehl: ")

    if eingabe == "ende":
        break

    print("Du hast", eingabe, "eingegeben.")

# da python von oben nach unten gelesen wird muss
# die methode zuerst definiert und dann aufgerufen werden
# zB kennt der Java-Compiler beim kompilieren bereits alle Methoden
def heileSpieler(lebenspunkte):
    lebenspunkte + 20

print("Lebenspunkte", lebenspunkte)

if lebenspunkte <= 0:
    heileSpieler(lebenspunkte)

```

```
    print("du wurdest gerade geheilt")

#wie in java beginnt ein Array bei 0 zu zählen
inventar = ["Schwert", "Heiltrank", "Schild"]
print(inventar[1])

inventar.append("Fleisch")
inventar.remove("Schwert")
print(inventar)

# key - value wird in Dictionary gespeichert
spieler = {
    "name": "julia",
    "alter": 18
}

print(spieler["name"])

# wenn bei try ein fehler passiert, wird except ausgeführt
# finally wird immer ausgeführt
try:
    zahl = int(input("Zahl eingeben: "))
except:
    print("das war keine zahl")
finally:
    print("Vorgang beendet")
```